

EgMoney

منصة تبادل العملات الرقمية

الخطة الشاملة لإكمال المشروع

وثيقة هندسية تفصيلية تحدد جميع الميزات المتبقية والمتطلبات التقنية اللازمة لتحويل منصة EgMoney إلى منصة إنتاجية جاهزة للنشر الفعلي، مع التزام صارم بالسلامة المالية وجودة الكود وتناسق الأنواع بين الواجهة والخادم.

الأولوية: إنتاجية

الحالة: نشطة

التاريخ: يونيو 2026

الإصدار: 2.0

فهرس المحتويات

8	+45	18	5
جداول قاعدة بيانات جديدة	ملف للتطوير/التعديل	ميزة رئيسية	مراحل تطوير
3			
مراحل مكتملة			

المراحل المكتملة (المراحل 1-3)

تم	إصلاح أخطاء KYC والمعاملات + محرك المطابقة + نظام حجز الأموال
----	---

المرحلة الرابعة: الأمان والتحقق

4.1	تأكيد البريد الإلكتروني عند التسجيل
4.2	المصادقة الثنائية (2FA/TOTP)
4.3	رفع ملفات KYC الفعلية
4.4	إدارة الجلسات وتجديد الرمز

المرحلة الخامسة: البنية التحتية وجودة الكود

5.1	طبقة API المركزية (Frontend + Admin)
5.2	مكتبة المكونات المشتركة
5.3	التحقق من المدخلات (Backend + Frontend)
5.4	إشعارات WebSocket الفورية للأوردرات

المرحلة السادسة: تحسينات التداول

6.1	رسوم التداول والعمولات
6.2	تكامل مكتبة lightweight-charts

6.3 واجهة الإداع في صفحة المحفظة

6.4 ترقيم الصفحات والتصفيه للتاريخ

المرحلة السابعة: تجربة المستخدم

7.1 واجهة تقديم KYC للمستخدم

7.2 نظام الإشعارات

7.3 صفحة سجلات التدقيق للإدارة

7.4 مكونات UI احترافية (Modals, Alerts)

المرحلة الثامنة: التجهيز للإنتاج

8.1 إدارة المتغيرات البيئية و env.example

8.2 Redis لمحدد المعدل والتخزين المؤقت

8.3 ترقيم إصدارات API

8.4 الاختبارات (وحدة + تكامل + E2E)

8.5 CI/CD وبرامج النشر

4 المرحلة الرابعة: الأمان والتحقق

هذه المرحلة تركز على تعزيز أمان المنصة بشكل جذري. بدون تأكيد البريد الإلكتروني والمصادقة الثنائية، لا يمكن اعتبار المنصة آمنة للاستخدام الإنتاجي مع أموال حقيقية. كل ميزة في هذه المرحلة تعتبر **حرجة** للأمان.

4.1 تأكيد البريد الإلكتروني عند التسجيل

حالياً يمكن لأي مستخدم التسجيل والبدء بالتداول فوراً دون تأكيد بريده الإلكتروني. هذا يفتح الباب أمام الحسابات المزيفة والبريد المزعج. يجب تعديل مسار التسجيل ليتطلب تأكيد البريد قبل تفعيل الحساب.

التغييرات في قاعدة البيانات

- إضافة حقل لنموذج `User: EmailVerified bool `gorm:"default:false" json:"email_verified"``
- نموذج جديد `EmailVerificationToken`: مماثل لـ `PasswordResetToken`، يحتوي على `UserID`, `Token` (64 حرف عشوائي), `ExpiresAt` (24 ساعة), `Used`

التغييرات في الخادم (Backend)

handlers/auth.go

- **Register**: بعد إنشاء المستخدم بنجاح، توليد رمز تحقق عشوائي 64 بايت وحفظه في `EmailVerificationToken`، ثم إرسال بريد يحتوي على رابط التحقق (مثال: `https://eg-money.local/verify-email?token=xxx`). المستخدم يُنشأ بحالة `EmailVerified=false`. لا يتم تسجيل الدخول تلقائياً بعد التسجيل.
- **VerifyEmail** (مسار جديد `(GET /api/auth/verify-email)`): مسار عام يستقبل `token` من `query parameter`. يبحث عن الرمز في قاعدة البيانات، يتحقق من عدم انتهاء الصلاحية وعدم الاستخدام السابق. إذا صالح: يحدّث `User.EmailVerified=true` ويحذف الرمز. يُرجع رسالة نجاح أو خطأ مناسب.
- **ResendVerification** (مسار جديد `(POST /api/auth/resend-verification)`): يستقبل البريد الإلكتروني، يبحث عن المستخدم، إذا لم يكن مؤكداً يُنشئ رمزاً جديداً ويرسله. معدل محدود: طلب واحد كل 60 ثانية.
- **Login**: إضافة تحقق: إذا `user.EmailVerified == false` يُرجع خطأ 403 مع رسالة "يرجى تأكيد بريدك الإلكتروني أولاً".

email/email.go

- إضافة دالة `SendVerificationEmail(to, token string)` بقوالب HTML احترافية تحمل شعار EgMoney وزر تأكيد واضح.

main.go

- إضافة المسارات العامة الجديدة: `GET /api/auth/verify-email` و `POST /api/auth/resend-verification`

التغييرات في الواجهة (Frontend)

register/page.tsx

- بعد التسجيل الناجح، عرض رسالة "تم إرسال رابط التأكيد إلى بريدك" بدلاً من تسجيل الدخول التلقائي.
- إضافة رابط "إعادة إرسال رابط التأكيد" يعمل بعد 60 ثانية.

verify-email/page.tsx (صفحة جديدة)

- يستقبل token من URL، يستدعي `GET /api/auth/verify-email?token=xxx`.
- عرض حالة النجاح مع زر "تسجيل الدخول" أو حالة الخطأ مع خيار إعادة الإرسال.

login/page.tsx

- إذا أرجع الخادم خطأ 403 (بريد غير مؤكد)، عرض رسالة واضحة مع زر "إعادة إرسال رابط التأكيد".

تنبيه أمني

رمز التحقق يجب أن يكون عشوائياً بتشفير آمن (`crypto/rand`) وليس رياضياً (`math/rand`). مدة الصلاحية 24 ساعة كحد أقصى. لا يجوز كشف ما إذا كان البريد مسجلاً أم لا في API إعادة الإرسال (نفس الرسالة دائماً).

4.2 المصادقة الثنائية (2FA/TOTP)

توفر المصادقة الثنائية طبقة حماية إضافية ضد اختراق كلمات المرور. نستخدم معيار TOTP (Time-based One-Time Password) المتوافق مع تطبيقات Google Authenticator و Authy. المصادقة الثنائية اختيارية للمستخدمين ولكنها إلزامية للإدارة عند سحب الأموال.

التغييرات في قاعدة البيانات

- إضافة حقول لنموذج `User`: `TwoFASecret string `gorm:"size:100" json:"-`` و `TwoFAEnabled bool `gorm:"default:false" json:"two_fa_enabled`` و `TwoFABackupCodes string `gorm:"type:text" json:"-`` (رموز طوارئ مشفرة JSON)

التغييرات في الخادم (Backend)

go.mod

- إضافة مكتبة: `github.com/pquerna/otp` (مكتبة TOTP القياسية في Go)

handlers/auth.go - التعديلات

- **Login**: بعد التحقق من كلمة المرور، إذا `TwoFAEnabled=true` : لا يُرجع الرمز المميز (JWT) مباشرة. بل يُرجع `{success: true, requires_2fa: true, temp_token: "xxx"}` حيث `temp_token` صالح لمدة 5 دقائق فقط.
- **Verify2FA (مسار جديد POST /api/auth/verify-2fa)**: يستقبل `temp_token` ورمز TOTP (6 أرقام). يتحقق من صحة `temp_token` ثم يتحقق من رمز TOTP باستخدام المكتبة. إذا صحيح: يُرجع الرمز المميز (JWT) الكامل. يتحقق أيضاً من رموز الطوارئ الاحتياطية.

handlers/twofa.go (ملف جديد)

- **Setup2FA (POST /api/user/2fa/setup)**: يولد سر TOTP فريد للمستخدم باستخدام `totp.Generate()` ، يحفظه في `User.TwoFASecret` (بدون تفعيل بعد)، يُرجع QR code URL (`otpauth://totp/...`) لعرضه في الواجهة.
- **Enable2FA (POST /api/user/2fa/enable)**: يستقبل رمز TOTP للتأكيد. إذا صحيح: يفعّل `TwoFAEnabled=true` ويولد 10 رموز طوارئ أحادية الاستخدام ويحفظها مشفرة. يُرجع الرموز الاحتياطية مرة واحدة فقط.
- **Disable2FA (POST /api/user/2fa/disable)**: يتطلب كلمة المرور الحالية + رمز TOTP. يوقف `TwoFAEnabled=false` ويمسح السر والرموز الاحتياطية.

- **GenerateBackupCodes (POST /api/user/2fa/backup-codes)**: يتطلب كلمة المرور + رمز

TOTP. يولد مجموعة جديدة من رموز الطوارئ (يُبطل القديمة).

handlers/wallet.go - التعديلات

- **WithdrawCurrency**: إذا `TwoFAEnabled=true` ، يتطلب رمز TOTP إضافي في الطلب

`two_fa_code` . التحقق منه قبل معالجة السحب.

التغييرات في الواجهة (Frontend)

login/page.tsx - التعديلات

- إذا أُرِجِع الخادم `requires_2fa: true` ، عرض حقل إدخال رمز TOTP بدلاً من إكمال تسجيل الدخول.

dashboard/profile/page.tsx - التعديلات

- إضافة قسم "المصادقة الثنائية": زر "تفعيل" يعرض QR code + حقل تأكيد الرمز. إذا مفعّل: عرض حالة "مفعّل" مع زر "تعطيل" وزر "إعادة إنشاء رموز الطوارئ".
- حفظ رموز الطوارئ عند التفعيل الأول وعرض تحذير "احفظ هذه الرموز في مكان آمن".

dashboard/wallet/page.tsx - التعديلات

- نافذة السحب: إذا المستخدم فَعّل 2FA ، إضافة حقل إلزامي لرمز TOTP قبل تأكيد السحب.

ملاحظة تقنية

temp_token المستخدم في تسجيل الدخول مع 2FA يجب أن يُخزن في Redis (أو قاعدة بيانات) مع TTL = 5 دقائق لمنع إعادة الاستخدام. في المرحلة الحالية يمكن تخزينه مؤقتاً في الذاكرة مع mutex حتى يتم إضافة Redis في المرحلة 8.

4.3 رفع ملفات KYC الفعلية

حالياً يقبل الخادم فقط نص URL للوثيقة (`document_url` كنص) دون رفع ملف فعلي. يجب توفير مسار رفع ملفات حقيقي يقبل صور الوثائق (جواز سفر، بطاقة هوية) ويحفظها بأمان.

التغييرات في الخادم (Backend)

handlers/kyc.go - التعديلات

- **UploadKYCDocument (مسار جديد POST /api/kyc/upload)**: يستقبل ملف multipart/form-data

التحقق من: نوع الملف (jpg, png, pdf فقط)، الحجم الأقصى (10MB). يحفظ الملف في

`./uploads/kyc/{user_id}_{timestamp}.{ext}` . يُرجع URL الملف النسبي.

- **SubmitKYC**: تعديل ليستقبل `document_url` ك URL الملف المرفوع (من المسار السابق) وليس نصاً حراً. التحقق من أن URL يبدأ بـ `/uploads/kyc/` .

`main.go`

- إضافة المسار: `POST /api/kyc/upload` (محمي بـ `AuthMiddleware`)

التغييرات في الواجهة (Frontend)

`dashboard/kyc/page.tsx` (صفحة جديدة - تفصيل في 7.1)

- مكون رفع ملف مع معاينة الصورة قبل الإرسال. استدعاء `POST /api/kyc/upload` أولاً ثم `POST /api/kyc/submit` مع URL المسترجع.

التغييرات في لوحة الإدارة (Admin)

`dashboard/kyc/page.tsx` - التعديلات

- عرض صورة/ملف الوثيقة المرفوعة ك رابط قابل للتحميل أو معاينة مباشرة (`img tag` أو `iframe` للـ PDF).

4.4 إدارة الجلسات وتجديد الرمز المميز

حالياً الرمز المميز (JWT) صالح لمدة 24 ساعة بدون آلية تجديد. عند انتهاء الصلاحية يفقد المستخدم كل سياقه ويجب إعادة تسجيل الدخول. هذا سيئ للتجربة ويخلق ثغرة أمنية لأن الرمز المسروق يظل صالحاً لمدة طويلة.

الاستراتيجية: Access Token + Refresh Token

النوع	المدة	التخزين	الاستخدام
Access Token	15 دقيقة	localStorage	جميع طلبات API
Refresh Token	7 أيام	قاعدة البيانات + httpOnly cookie	تجديد Access Token فقط

التغييرات في قاعدة البيانات

- نموذج جديد RefreshToken: ID, UserID (فهرس), Token (فهرس فريد, 64 حرف), UserAgent, IPAddress, ExpiresAt, Revoked (bool), CreatedAt

التغييرات في الخادم (Backend)

handlers/auth.go - التعديلات

- **Login/Verify2FA:** بالإضافة إلى Access Token (15 دقيقة)، توليد Refresh Token (64 بايت عشوائي) وحفظه في قاعدة البيانات. إرجاع Access Token في JSON و Refresh Token ك httpOnly cookie آمن.
- **RefreshToken (مسار جديد POST /api/auth/refresh):** يقرأ Refresh Token من الـ cookie. يبحث عنه في قاعدة البيانات. يتحقق من عدم إلغائه وعدم انتهاء الصلاحية. إذا صالح: يولد Access Token جديد (15 دقيقة) ويُرجعه.
- **Logout (مسار جديد POST /api/auth/logout):** يلغي Refresh Token الحالي في قاعدة البيانات ويمسح الـ cookie.
- **JWT Access Token:** تغيير المدة من 24 ساعة إلى 15 دقيقة في دالة توليد الرمز.

handlers/middleware.go - التعديلات

- **AuthMiddleware:** إذا انتهت صلاحية Access Token، يُرجع: `{success: false, message: "الرمز المميز قد انتهت صلاحية", code: "TOKEN_EXPIRED"}`. الكود المميز `TOKEN_EXPIRED` يسمح

لواجهة بالتمييز بين انتهاء الصلاحية والرمز غير الصالح.

التغييرات في الواجهة (Frontend + Admin)

lib/api.ts (ملف جديد - سيتم تفصيله في 5.1)

- معترض (interceptor) HTTP: عند استلام خطأ 401 مع `code: "TOKEN_EXPIRED"` ، يستدعي تلقائياً `POST /api/auth/refresh` ثم يعيد الطلب الأصلي بالرمز الجديد. إذا فشل التجديد أيضاً: توجيهه إلى صفحة تسجيل الدخول.
- تحديث `localStorage` بالرمز الجديد بعد كل تجديد ناجح.

قواعد أمنية إلزامية

- Refresh Token يجب أن يكون `httpOnly + Secure + SameSite=Strict` cookie. لا يجوز تخزينه في `localStorage`.
- كل تجديد ناجح للرمز يُلغي Refresh Token القديم ويولد واحداً جديداً (Rotation). هذا يمنع إعادة الاستخدام إذا سُرق الرمز.
- تسجيل الخروج من جميع الأجهزة: مسار `POST /api/auth/logout-all` يلغي جميع Refresh Tokens للمستخدم.

5 المرحلة الخامسة: البنية التحتية وجودة الكود

هذه المرحلة تعالج المشاكل الهيكلية في الكود الحالي: تكرار استدعاءات API في كل صفحة، غياب مكتبة مكونات مشتركة، ضعف التحقق من المدخلات، وغياب الإشعارات الفورية. إصلاح هذه المشاكل سيُحسّن إنتاجية التطوير بشكل كبير ويمنع أخطاء مستقبلية.

5.1 طبقة API المركزية

حالياً كل صفحة في الواجهة ولوحة الإدارة تكرر نفس كود `fetch` مع إضافة الرمز المميز ومعالجة الأخطاء. يجب إنشاء طبقة API مركزية تُوحّد جميع الاستدعاءات وتتعامل تلقائياً مع تجديد الرمز المميز ومعالجة الأخطاء.

الملفات الجديدة

`frontend/src/lib/api.ts + admin/src/lib/api.ts`

- فئة `ApiClient`: singleton يحتوي على `baseUrl`, `token getter/setter` و `refresh mechanism`.
- الدوال الأساسية: `get(path, params?)`, `post(path, body)`, `put(path, body)`, `del(path)`
- معترض تلقائي: يضيف `Authorization: Bearer {token}` لكل طلب. عند 401 `TOKEN_EXPIRED`: يحاول التجديد ثم يعيد الطلب. عند فشل التجديد: يسمح الرمز ويُوّجه لتسجيل الدخول.
- معالجة الأخطاء الموحدة: تحويل جميع الاستجابات إلى `{success, data, message}` بشكل متسق.

`frontend/src/lib/endpoints.ts + admin/src/lib/endpoints.ts`

- تصدير دوال مُسمّى لكل مسار `API`: `auth.login()`, `auth.register()`, `exchange.placeOrder()`, `wallet.getBalances()`, `admin.getUsers()` ... إلخ
- أنواع `TypeScript`: تحديد واجهات (interfaces) لكل طلب واستجابة مع أنواع دقيقة.

مثال على البنية

```
// lib/api.ts
class ApiClient {
  private baseUrl: string;
  async get<T>(path: string): Promise<ApiResponse<T>>
  async post<T>(path: string, body: any): Promise<ApiResponse<T>>
  private async refreshToken(): Promise<boolean>
}
```

```
export const api = new ApiClient();

// lib/endpoints.ts
export const auth = {
  login: (data: LoginReq) => api.post<LoginRes>('/auth/login', data),
  register: (data: RegisterReq) => api.post<RegisterRes>('/auth/register', data),
}
export const exchange = {
  placeOrder: (data: OrderReq) => api.post<Order>('/exchange/order', data),
  getOrders: (params?: OrderParams) => api.get<Order[]>('/exchange/orders'),
}
```

5.2 مكتبة المكونات المشتركة

الواجهة ولوحة الإدارة تكرر نفس أنماط (glass-panel, btn-primary, input-field) CSS دون مكونات React قابلة لإعادة الاستخدام. هذا يُصعّب الصيانة ويؤدي إلى تناقضات بصرية.

المكونات المطلوبة

المكون	الاستخدام	الخصائص
GlassCard	حاوية شفافة أساسية	?title?, children, className?, actions
DataTable	جدول بيانات مع ترقيم صفحات	columns, data, pagination?, loading?, ?onRowClick
Modal	نافذة منبثقة (بديل prompt)	?isOpen, onClose, title, children, footer
InputField	حقل إدخال مع تسمية وخطأ	label, error?, type?, icon?, ...inputProps
Button	زر موحد (primary/danger/ghost)	variant, loading?, disabled?, onClick, children
StatusBadge	شارة حالة ملونة	?status, label
ConfirmDialog	حوار تأكيد (بديل confirm)	isOpen, onConfirm, onCancel, ?message, variant
EmptyState	حالة فارغة مع أيقونة	?icon, title, description
Loader	مؤشر تحميل	?size?, text

/frontend/src/components/ui/ + admin/src/components/ui

- كل مكون في ملف منفصل مع أنواع TypeScript كاملة.
- بعد الإنشاء، استبدال جميع استخدامات `prompt()` و `confirm()` بـ `Modal` و `ConfirmDialog`.

5.3 التحقق من المدخلات (Backend + Frontend)

حالياً الخادم يعتمد على تحقق بسيط جداً (مثل طول العنوان الأدنى 10 أحرف للسحب). هذا غير كافٍ لمنصة مالية. يجب تطبيق تحقق شامل على كلا الجانبين.

قواعد التحقق المطلوبة

الحقل	القاعدة	المسار
البريد الإلكتروني	صيغة صالحة + طول 3-255 + نطاق موجود	Register, ForgotPassword
اسم المستخدم	3-30 حرف، أحرف إنجليزية + أرقام + _ فقط	Register
كلمة المرور	+8 حرف، حرف كبير + صغير + رقم + رمز	Register, ChangePassword, ResetPassword
الكمية (Order)	رقم موجب، دقة 8 خانات عشرية كحد أقصى، حد أدنى حسب الزوج	PlaceOrder
السعر (Limit)	رقم موجب، دقة 8 خانات عشرية كحد أقصى	PlaceOrder (LIMIT/STOP_LIMIT)
رمز الزوج (Symbol)	يجب أن يكون من القائمة المعتمدة (8 أزواج)	PlaceOrder
عنوان السحب	طول 20-500، تحقق تنسيق حسب العملة (مستقبل: تحقق blockchain)	WithdrawCurrency
مبلغ السحب	موجب، لا يتجاوز الرصيد المتاح (المحجوز محسوب)، حد أدنى حسب العملة	WithdrawCurrency

التنفيذ في الخادم

handlers/validation.go (ملف جديد)

دوال التحقق: `ValidateEmail()` , `ValidateUsername()` , `ValidatePassword()`

`ValidateOrderInput()` , `ValidateWithdrawal()`

كل دالة تُرجع قاموس أخطاء مفصل: `map[string]string` (اسم الحقل => رسالة الخطأ)

دمجها في كل handler قبل معالجة الطلب

التنفيذ في الواجهة

- نفس قواعد التحقق مكتوبة بـ TypeScript لتعمل في المتصفح قبل إرسال الطلب.
- تكامل مع مكون `InputField` لعرض أخطاء فورية أثناء الكتابة.

5.4 إشعارات WebSocket الفورية للأوردرات

حالياً عندما يتم تنفيذ أوردرد في محرك المطابقة (matching engine)، لا يتلقى المستخدم أي إشعار فوري. يجب عليه تحديث الصفحة يدوياً. هذا غير مقبول في منصة تداول حقيقية حيث الثواني تهم.

التصميم: WebSocket خاص بالمستخدم

تحديث الواجهة ← للمستخدم WS ← Hub بث عبر ← تنفيذ الأوردرد ← محرك المطابقة

التغييرات في الخادم (Backend)

websocket/hub.go (ملف جديد)

- **UserHub**: مركز اتصالات مركزي يدير اتصالات WebSocket للمستخدمين. يحتوي على خريطة `map[uint][]*UserClient` (`UserID` ≤ قائمة العملاء).
- **UserClient**: يحتوي على معرف المستخدم وقناة إرسال (channel).
- **NotifyUser(userID, event)**: دالة عامة تنشر حدثاً لجميع اتصالات المستخدم. الحدث يحتوي على:


```
{type: "order_fill"|"order_trigger"|"deposit_approved"|"withdrawal_processed", data: {...}}
```

websocket/user_ws.go (ملف جديد)

- **HandleUserWebSocket**: مسار `GET /ws/user` محمي بـ `AuthMiddleware`. يصادق المستخدم من query parameter token. يُسجّل الاتصال في `UserHub`. يقرأ من قناة الإرسال ويُرسل الرسائل كـ JSON. ينظف عند قطع الاتصال.

matching/execution.go - التعديلات

- بعد كل `ExecuteFill()` ناجح، استدعاء `websocket.NotifyUser(order.UserID, OrderFillEvent{...})`
- بعد كل `processTriggerOrder()` ناجح، إشعار المستخدم بتفعيل الأوردرد الشرطي.

التعديلات - handlers/admin.go

- بعد `ReviewTransaction()` ، إشعار المستخدم: `NotifyUser(tx.UserID, WithdrawalProcessedEvent{...})` أو `DepositApprovedEvent{...})`
- بعد `ReviewKYC()` ، إشعار المستخدم بحالة KYC.

التغييرات في الواجهة (Frontend)

التعديلات - dashboard/layout.tsx

- الاتصال بـ `wss://api.eg-money.local/ws/user?token=xxx` عند تسجيل الدخول.
- عند استلام إشعار: عرض toast إشعار مع صوت تنبيه اختياري.
- تحديث بيانات الصفحة تلقائياً (أرصدة، أوامر) عند استلام أحداث ذات صلة.

6 المرحلة السادسة: تحسينات التداول

هذه المرحلة تضيف الميزات المالية والتداولية الأساسية الناقصة: رسوم العمولة، رسوم بيانية احترافية، واجهة الإيداع، وترقيم الصفحات. بدون رسوم تداول لا توجد إيرادات للمنصة. بدون واجهة إيداع لا يمكن للمستخدم إضافة أموال.

6.1 رسوم التداول والعمولات

المنصة حالياً لا تخصم أي رسوم على التداول. هذا يعني عدم وجود نموذج إيرادات. يجب تطبيق نظام رسوم شامل مع دعم رسوم مختلفة حسب نوع الأوردر ومستوى المستخدم.

التغييرات في قاعدة البيانات

- **إضافة حقول لنموذج Order:** `Fee float64 `gorm:"default:0;type:decimal(36,18)"` و `FeeCurrency string `gorm:"size:20" json:"fee_currency"```
- **نموذج جديد FeeSchedule:** ID, UserType (USER/VERIFIED_USER), OrderType (MARKET/LIMIT/STOP_LIMIT/TAKE_PROFIT), MakerFee (float64), TakerFee (float64), MinFee (float64), CreatedAt, UpdatedAt

هيكل الرسوم

نوع المستخدم	نوع الأوردر	رسوم الصانع (Maker)	رسوم الآخذ (Taker)
USER	MARKET	-	0.25%
USER	LIMIT (fill)	0.15%	0.20%
VERIFIED_USER	MARKET	-	0.15%
VERIFIED_USER	LIMIT (fill)	0.10%	0.12%

التغييرات في الخادم (Backend)

matching/execution.go - التعديلات

- **ExecuteFill:** حساب الرسوم بعد التنفيذ: `fee = fillAmount * fillPrice * feeRate`. خصم الرسوم من العملة المتلقاة. مثال: شراء BTC بسعر 50000 بكمية 0.1 = 5000 USDT. رسوم 0.25% = 12.5 USDT. بدلاً من تلقي 0.1 BTC كاملة، يتلقى 0.099975 BTC والـ 12.5 USDT تُخصم من محفظة USDT أو تُضاف لمحفظة الرسوم.

- **ReserveFunds**: يجب أن يحجز مبلغاً إضافياً لتغطية الرسوم المتوقعة: `reservedAmount =`

`orderCost + estimatedFee`

- تحديث `Order.Fee` و `Order.FeeCurrency` عند التنفيذ.

handlers/admin.go - إضافات

- **مسارات إدارة الرسوم**: `GET /api/admin/fees` (عرض جدول الرسوم), `PUT`

`/api/admin/fees/:id` (تعديل نسبة الرسوم).

التغييرات في الواجهة (Frontend)

dashboard/exchange/page.tsx - التعديلات

- نموذج الطلب: عرض الرسوم المقدرة بجانب التكلفة الإجمالية. "الرسوم المقدرة: ~0.25% = 12.50 USDT"

- قائمة الأوامر: عرض الرسوم الفعلية المخصومة لكل أوامر منفذ.

قاعدة مالية حرجة

حساب الرسوم يجب أن يتم داخل نفس معاملة قاعدة البيانات (DB Transaction) التي تنفذ الأوامر. لا يجوز حساب الرسوم في معاملة منفصلة لأن هذا قد يؤدي إلى تنفيذ الأوامر بدون خصم الرسوم في حالة فشل المعاملة الثانية. الرسوم تُحسب من المبلغ المحجوز (ReservedAmount) وليس من رصيد منفصل.

6.2 | تكامل مكتبة lightweight-charts

حالياً صفحة التداول تستخدم رسم Canvas يدوي للشموع اليابانية. مكتبة `lightweight-charts` مُدرجة في `package.json` لكنها غير مستخدمة. هذه المكتبة توفر رسوماً بيانية احترافية مع دعم أصلي للتكبير والتصغير والمؤشرات الفنية.

dashboard/exchange/page.tsx - إعادة كتابة قسم الرسم البياني

- استبدال كود Canvas اليدوي بـ `createChart()` من `lightweight-charts`.

- إضافة `CandlestickSeries` مع بيانات `klines` من `Binance`.

- إضافة `VolumeSeries` (أعمدة حجم التداول) أسفل الشموع.

- دعم تبديل الأطر الزمنية (1د، 5د، 15د، ساعة، 4 ساعات، يوم).

- تحديث البيانات في الوقت الفعلي عبر `WebSocket`.

- إضافة خطوط السعر الحالي (price line) وخطوط الأوامر المعلقة (order lines).

6.3 واجهة الإيداع في صفحة المحفظة

حالياً صفحة المحفظة تعرض فقط زر السحب بدون أي واجهة للإيداع. المسار `POST /api/wallet/deposit` موجود في الخادم لكن لا توجد واجهة لاستدعائه. يجب إضافة نموذج إيداع كامل.

dashboard/wallet/page.tsx - التعديلات

- إضافة زر "إيداع" بجانب كل عملة في بطاقات المحفظة.
- نافذة إيداع (Modal): اختيار العملة، إدخال المبلغ، حقل ملاحظات اختياري.
- استدعاء `POST /api/wallet/deposit` وعرض رسالة "تم إنشاء طلب الإيداع، بانتظار موافقة الإدارة".
- عرض حالة طلبات الإيداع المعلقة في قسم المعاملات.

6.4 ترقيم الصفحات والتصفية للتاريخ

صفحة السجل (History) تجلب جميع الأوامر دفعة واحدة بدون ترقيم صفحات أو تصفية. هذا سيء للأداء مع كثرة البيانات. الخادم يدعم ترقيم الصفحات عبر `getPagination()` لكن الواجهة لا تستخدمه.

dashboard/history/page.tsx - إعادة كتابة

- إضافة معلمات ترقيم الصفحات: `?page=1&limit=20`
- إضافة تصفية حسب: نوع الأوامر (MARKET/LIMIT/STOP_LIMIT/TAKE_PROFIT)، الجانب (BUY/SELL)، الحالة (PENDING/FILLED/CANCELLED)، الزوج، تاريخ من-إلى.
- مكون ترقيم صفحات مرئي في أسفل الجدول.
- تحديث الخادم: إضافة معلمات تصفية إلى `GetOrders` (symbol, side, type, status, date_from, date_to).

7 المرحلة السابعة: تجربة المستخدم

هذه المرحلة تُكمل الميزات الناقصة في واجهة المستخدم: واجهة تقديم KYC، نظام إشعارات، صفحة سجلات التدقيق للإدارة، ومكونات UI احترافية. كل ميزة هنا تعزز ثقة المستخدم وكفاءة الإدارة.

7.1 واجهة تقديم KYC للمستخدم

المسار `POST /api/kyc/submit` موجود في الخادم لكن لا توجد صفحة في الواجهة لتقديم طلب التحقق. يجب إنشاء صفحة كاملة تسمح للمستخدم برفع وثائقه وتتبع حالة الطلب.

dashboard/kyc/page.tsx (صفحة جديدة)

- **الحالة الأولية (لم يقدم):** نموذج يتضمن: الاسم الكامل، نوع الوثيقة (جواز سفر / بطاقة هوية / رخصة قيادة)، رقم الوثيقة، رفع صورة الوثيقة (أمام + خلف)، رفع صورة شخصية مع الوثيقة.
- **رفع الملفات:** استدعاء `POST /api/kyc/upload` لكل ملف ثم `POST /api/kyc/submit` مع جميع البيانات.
- **الحالة المعلقة (PENDING):** عرض رسالة "طلبك قيد المراجعة" مع تاريخ التقديم.
- **الحالة المقبولة (APPROVED):** عرض شارة "هوية موثقة" مع تاريخ القبول.
- **الحالة المرفوضة (REJECTED):** عرض سبب الرفض مع إمكانية إعادة التقديم.

dashboard/layout.tsx - التعديلات

- إضافة رابط "التحقق من الهوية" في القائمة الجانبية بين المحفظة والملف الشخصي.
- عرض شارة بجانب اسم المستخدم تشير لحالة KYC (رمادية/صفراء/خضراء/حمراء).

7.2 نظام الإشعارات

المنصة لا تملك نظام إشعارات مركزي. يجب إنشاء نظام يخزن الإشعارات في قاعدة البيانات ويعرضها في الواجهة مع دعم الإشعارات الفورية عبر WebSocket.

التغييرات في قاعدة البيانات

- **نموذج جديد** Notification: ID, UserID (فهرس)، Type (order_fill, deposit_approved, withdrawal_processed, kyc_approved, kyc_rejected, security_alert), Title (string), Body

التغييرات في الخادم (Backend)

handlers/notification.go (ملف جديد)

- **GetNotifications (GET /api/notifications)**: يجلب آخر 50 إشعار مع دعم ترقيم الصفحات ومعلمة `unread_only`.
- **MarkAsRead (PUT /api/notifications/:id/read)**: يحدّث `Read=true`.
- **MarkAllAsRead (PUT /api/notifications/read-all)**: يحدّث جميع إشعارات المستخدم.
- **GetUnreadCount (GET /api/notifications/unread-count)**: يُرجع عدد الإشعارات غير المقروءة.

models/notification.go (ملف جديد) + التكامل

- في كل handler يُنشئ حدثاً (تنفيذ أورد، موافقة إيداع، رفض KYC): إنشاء سجل Notification مرتبط بالمستخدم + بث عبر WebSocket.

التغييرات في الواجهة (Frontend)

dashboard/layout.tsx + components/NotificationBell.tsx

- أيقونة جرس في الشريط العلوي مع عداد الإشعارات غير المقروءة.
- قائمة منسدلة تعرض آخر 10 إشعارات مع ألوان مختلفة حسب النوع.
- صفحة إشعارات كاملة: `dashboard/notifications/page.tsx`

7.3 صفحة سجلات التدقيق للإدارة

لوحة الإدارة تعرض بعض سجلات التدقيق في الصفحة الرئيسية فقط بدون صفحة مخصصة. يجب إنشاء صفحة مخصصة مع تصفية وبحث متقدم.

admin/dashboard/audit-logs/page.tsx (صفحة جديدة)

- جدول كامل لسجلات التدقيق مع تصفية حسب: نوع الإجراء، المستخدم، التاريخ.
- بحث نصي في حقل التفاصيل.
- ترقيم صفحات.
- تصدير CSV لسجلات التدقيق.

admin/dashboard/layout.tsx - التعديلات

7.4 مكونات UI احترافية (Modals, Alerts)

عدة صفحات تستخدم `prompt()` و `confirm()` الأصليين في المتصفح وهو غير احترافي. يجب استبدالها بمكونات React مخصصة.

الصفحة	الاستخدام الحالي	الاستبدال
KYC (Admin)	<code>prompt()</code> لسبب الرفض	Modal مع حقل نصي
Transactions (Admin)	<code>prompt()</code> لـ TxID	Modal مع حقل إدخال
Exchange (Frontend)	<code>confirm()</code> لإلغاء الأوردر	ConfirmDialog
Wallet (Frontend)	<code>confirm()</code> للسحب	ConfirmDialog مع ملخص

8 المرحلة الثامنة: التجهيز للإنتاج

المرحلة الأخيرة تُجهّز المنصة للنشر الإنتاجي الفعلي. تشمل: إدارة آمنة للمتغيرات البيئية، Redis للأداء والأمان، ترقيم إصدارات API، اختبارات شاملة، وخط أنابيب CI/CD.

8.1 إدارة المتغيرات البيئية

حالياً العديد من القيم الحساسة مُرمّزة في `docker-compose.yml` (كلمة مرور قاعدة البيانات، سر JWT الافتراضي، كلمة مرور الإدارة). يجب نقلها جميعاً إلى ملف `env` مع توثيق كامل.

`env.example` (ملف جديد في جذر المشروع).

- توثيق جميع المتغيرات المطلوبة مع قيم افتراضية آمنة وتعليقات توضيحية.
- المتغيرات الحرجة: `DATABASE_URL`, `JWT_SECRET`, `ADMIN_PASSWORD`, `SMTP_HOST/PORT/USER/PASS`
- المتغيرات الاختيارية: `REDIS_URL`, `PORT`, `SMTP_FROM_NAME/EMAIL`
- تحديث `docker-compose.yml` لقراءة من ملف `env` بدلاً من القيم المُرمّزة.

gitignore - التحديث

- التأكد من وجود `.env` في `gitignore` (لا يجوز رفع القيم الحقيقية).
- رفع `env.example` فقط كقالب.

قاعدة أمنية

`JWT_SECRET` يجب أن يكون +32 حرفاً عشوائياً في الإنتاج. لا يجوز استخدام القيمة الافتراضية. يجب إنشاؤه بأمر مثل: `openssl rand -hex 32`. كلمة مرور الإدارة الافتراضية يجب تغييرها فور أول تسجيل دخول.

8.2 Redis لمحدد المعدل والتخزين المؤقت

محدد المعدل الحالي يعمل في الذاكرة فقط. هذا يعني: (1) يُفقد عند إعادة تشغيل الخادم، (2) لا يعمل عبر عدة نسخ (instances) من الخادم. Redis يحل كلا المشكلتين ويوفر تخزيناً مؤقتاً لأسعار السوق.

التغييرات في البنية التحتية

`docker-compose.yml` - إضافة خدمة Redis

- إضافة خدمة `redis:7-alpine` مع `volume` مؤقت و `health check`.

- إضافة متغير `REDIS_URL` لخدمة الخادم.

التغييرات في الخادم (Backend)

`go.mod`

- إضافة: `github.com/redis/go-redis/v9`

handlers/middleware.go - التعديلات

- **RateLimiter**: إذا Redis متاح، استخدامه بدلاً من الذاكرة المحلية. مفاتيح `rate_limit:{ip}` Redis: مع TTL تلقائي.

matching/engine.go - التعديلات

- **تخزين مؤقت للأسعار**: حفظ آخر سعر في Redis مع TTL. هذا يسمح لنسخ متعددة من الخادم بمشاركة بيانات الأسعار.

8.3 ترقيم إصدارات API

جميع المسارات حالياً تحت `/api/...` بدون رقم إصدار. هذا يُصعّب إجراء تغييرات جذرية على API بدون كسر التوافق مع الإصدارات السابقة. يجب إضافة بادئة الإصدار.

main.go - التعديلات

- تغيير جميع المسارات من `/api/...` إلى `/api/v1/...`.
- إضافة مسارات `/api/...` لإعادة توجيه (redirect) مؤقتة للتوافق مع الإصدارات السابقة لمدة انتقالية.
- تحديث `NEXT_PUBLIC_API_URL` في الواجهة ولوحة الإدارة.

8.4 الاختبارات (وحدة + تكامل + E2E)

المشروع لا يحتوي على أي اختبارات حالياً. لمنصة مالية هذا غير مقبول. يجب إنشاء اختبارات شاملة على ثلاثة مستويات.

اختبارات الوحدة (Backend)

`backend/*_test.go`

- **matching/execution_test.go**: اختبار `ReserveFunds`، `ExecuteFill`، `ReleaseFunds` مع سيناريوهات: رصيد كافٍ، رصيد غير كافٍ، حجز مزدوج (race condition)، استرداد بعد إلغاء.

- `handlers/validation_test.go`: اختبار جميع دوال التحقق من المدخلات.
- `handlers/auth_test.go`: اختبار التسجيل، تسجيل الدخول، التحقق من البريد، 2FA.

اختبارات التكامل (Backend)

`/backend/tests/integration`

- اختبار المسار الكامل: تسجيل → تأكيد البريد → تسجيل الدخول → إيداع → موافقة الإدارة → وضع أوردور → تنفيذ → سحب → موافقة الإدارة.
- اختبار التزامن: أوردوران متزامنان على نفس الرصيد. التحقق من أن الرصيد لا ينخفض تحت الصفر أبداً.
- قاعدة بيانات اختبارية معفية (test database) تُعاد تهيئتها قبل كل اختبار.

E2E (Frontend) اختبارات

`/frontend/e2e/ + admin/e2e`

- استخدام Playwright لاختبار المسارات الحرجة: التسجيل، تسجيل الدخول، وضع أوردور، السحب.
- اختبار سيناريوهات الخطأ: كلمة مرور خاطئة، رصيد غير كافٍ، رمز منتهي الصلاحية.

8.5 CI/CD وبرامج النشر

لا يوجد خط أنابيب للتكامل المستمر والنشر المستمر. يجب إنشاء واحد يضمن جودة الكود قبل دمجه.

`github/workflows/ci.yml` (ملف جديد)

- **عند Push/Pull Request**: تشغيل اختبارات الخادم (`go test`)، تشغيل اختبارات الواجهة (`npm run build`)، فحص الأمان (`go vet`, `npm audit`)، التحقق من التنسيق (`gofmt`, `prettier`).
- **عند Push إلى master**: بناء صور Docker تلقائياً ورفعها إلى `registry`.
- **النشر**: SSH إلى الخادم الإنتاجي، سحب الصور الجديدة، تشغيل `docker-compose up -d`.

ملخص الأولويات والجدول الزمني

مصفوفة الأولويات

المرحلة	الميزة	الأولوية	التعقيد	الملفات المقدرة
4.1	تأكيد البريد الإلكتروني	حرجة	متوسط	6 ملفات
4.2	المصادقة الثنائية 2FA	حرجة	عالي	8 ملفات
4.3	رفع ملفات KYC	عالية	منخفض	4 ملفات
4.4	إدارة الجلسات	حرجة	عالي	7 ملفات
5.1	طبقة API المركزية	عالية	متوسط	6 ملفات
5.2	مكتبة المكونات	متوسطة	متوسط	10 ملفات
5.3	التحقق من المدخلات	عالية	متوسط	5 ملفات
5.4	إشعارات WebSocket	عالية	عالي	6 ملفات
6.1	رسوم التداول	حرجة	عالي	7 ملفات
6.2	مكتبة الرسوم البيانية	متوسطة	متوسط	2 ملفات
6.3	واجهة الإيداع	عالية	منخفض	2 ملفات
6.4	ترقيم الصفحات	متوسطة	منخفض	3 ملفات
7.1	واجهة KYC للمستخدم	عالية	متوسط	3 ملفات
7.2	نظام الإشعارات	متوسطة	عالي	7 ملفات
7.3	سجلات التدقيق	منخفضة	منخفض	2 ملفات
7.4	مكونات UI	متوسطة	منخفض	5 ملفات
8.1	المتغيرات البيئية	عالية	منخفض	3 ملفات
8.2	Redis	متوسطة	متوسط	5 ملفات
8.3	ترقيم API	منخفضة	منخفض	4 ملفات
8.4	الاختبارات	عالية	عالي	+10 ملفات

3 ملفات	متوسط	متوسطة	CI/CD	8.5
---------	-------	--------	-------	-----

الجدول الزمني المقترح

المرحلة	المدة المقدرة	المخرجات
المرحلة 4 - الأمان	5-7 أيام	تأكيد بريد + 2FA + رفع KYC + تجديد الرمز
المرحلة 5 - البنية	4-5 أيام	API مركزي + مكونات + تحقق + WebSocket إشعارات
المرحلة 6 - التداول	4-5 أيام	رسوم + رسوم بيانية + إيداع + ترقيم صفحات
المرحلة 7 - تجربة المستخدم	3-4 أيام	KYC + إشعارات + سجلات + UI
المرحلة 8 - الإنتاج	5-7 أيام	CI/CD + env + Redis + API v1. + اختبارات

قاعدة التنفيذ

كل ميزة تُنفَّذ بالترتيب داخل كل مرحلة. بعد إكمال كل ميزة: (1) اختبار يدوي شامل، (2) commit + push إلى master، (3) انتقال للميزة التالية. لا يجوز الانتقال لميزة جديدة حتى تعمل الميزة الحالية بشكل كامل. كل تغيير مالي يجب أن يكون داخل DB Transaction مع Row Locking.

الجدول الجديدة المطلوبة في قاعدة البيانات

الجدول	المرحلة	الغرض
email_verification_tokens	4.1	رموز تأكيد البريد
refresh_tokens	4.4	رموز تجديد الجلسة
fee_schedules	6.1	جدول رسوم التداول
notifications	7.2	إشعارات المستخدمين

الحقول الجديدة في النماذج الموجودة

النموذج	الحقل	المرحلة
User	email_verified (bool)	4.1

